Feature Selection and Feature Engineering in Data Analytics

Introduction to Feature Engineering

- Feature engineering is the process of creating, transforming, and selecting variables (features) from raw data.
- It helps machine learning and analytics models understand patterns in the data more effectively.
- Well-designed features significantly improve model accuracy.

Why Feature Engineering is Important

- Raw data is rarely suitable for analytics models.
- Feature engineering converts raw data into meaningful variables.
- It improves prediction accuracy and model interpretability.
- It helps algorithms learn patterns more efficiently.

Purpose of Feature Engineering

- Improve predictive performance of models
- Reduce noise in raw datasets
- Transform variables into useful formats
- Extract hidden information from existing data
- Improve training speed and model stability

Significance in Data Analytics

- Feature engineering bridges the gap between raw data and machine learning models.
- It often contributes more to model performance than the choice of algorithm.
- Good features capture the real patterns present in the dataset.

Common Feature Engineering Techniques

- Handling missing values
- Encoding categorical variables
- Feature scaling and normalization
- Creating interaction features (new features created by combining two or more existing features)
- Polynomial features (new features created by raising existing features to powers or combining them to introduce no-linearity)
- Date and time feature extraction

Application Areas

- Financial risk prediction
- Customer behavior analysis
- Fraud detection
- Healthcare diagnostics
- Recommendation systems
- Sales forecasting

Numerical Example – Raw Dataset

- Suppose we have a dataset of house sales:
- Size (sq.ft): 1000, 1200, 1500
- Bedrooms: 2, 3, 3
- Price (₹ lakhs): 40, 50, 65
- Raw features may not fully capture useful patterns.

Feature Engineering Example

Create a new feature: Price per square foot

- Formula: Price / Size

- For house 1: 40 / 1000 = 0.04
- For house 2: 50 / 1200 = 0.0417
- For house 3: 65 / 1500 = 0.0433

Note: This new feature helps compare property value more effectively.

Another Example – Interaction Feature

Original features:

- Study Hours (X1) = 5
- Attendance (X2) = 80%

New feature created:

- Study Impact (X’) = Study Hours (X1) × Attendance (X2)
- Study Impact (X’) = 5 × 0.80 = 4

Note: This feature better represents learning engagement.

Benefits of Feature Engineering

- Improves prediction accuracy
- Helps algorithms detect patterns
- Reduces model complexity
- Enhances interpretability
- Handles real-world messy data

Challenges in Feature Engineering

- Requires domain knowledge
- Time-consuming process
- Risk of creating irrelevant features
- High dimensionality issues

Best Practices

- Understand the domain of the dataset
- Use statistical insights before transformation
- Avoid data leakage (occurs when information from outside the training dataset (especially future or target-related data) is used to build the model)
- Validate features using model performance
- Use feature selection techniques

Example: Polynomial feature engineering

X_1 (Size)	X_2 (Rooms)
1	2
2	3
3	4

❖ Apply Polynomial Feature Generation with degree = 2

Assume:

For 2 variables and degree 2:

$$[1, , X_1, , X_2, X_1^2, X_2^2, X_1X_2]$$

Note: Y=w1X1+w2X2 (Only Linear Relationship or without polynomial features)

Y=w1X1+w2X2+w3X12+w4X22+w5X1X2 (with polynomial features)

Feature Explosion

- When variables (features) becomes too large to effectively manage, leading to computational inefficiencies and model performance degradation.
- It may result in overfitting, increased latency and inconsistent result/performance
- Solution: Feature Selection, and Feature extraction using suitable method (using PCA and others).

Example: from numerical in last slide

For Number of features n = 2

Degree d = 2

$$\frac{(n+d)!}{n! \cdot d!} = \frac{(2+2)!}{2! \cdot 2!} = 6$$

Types of Feature Selection Techniques

1. Filter Methods

- These methods select features based on statistical properties, independent of any machine learning model.

Common Techniques:

- Correlation Coefficient (Measures linear relationship between feature X and target Y)
- Chi-Square Test (Dependence between feature and target)
- ANOVA (Analysis of Variance) (Compares means of multiple groups)
- Information Gain (Measures reduction in entropy after splitting)

Example:

- If two features have high correlation (e.g., 0.95), one can be removed as redundant.

Advantages:

- Fast and computationally efficient
- Works well with large datasets

Disadvantages:

- Usually ignores feature interaction

2. Wrapper Methods

- These methods evaluate feature subsets using a specific machine learning model.

Common Techniques:

- Forward Selection (Step-wise Addition)
- Backward Elimination (Step-wise Removal)
- Recursive Feature Elimination (RFE) (remove least important feature based on model importance)

Working:

- Try different combinations of features
- Select the subset that gives best model performance

Advantages:

- High accuracy
- Considers feature interactions

Disadvantages:

- Computationally expensive
- Slow for large datasets

Method	Start	Process	Criteria	Result
Forward Selection	No features	Add features	Accuracy	Builds model
Backward Elimination	All features	Remove features	p-value	Simplifies model
RFE	All features	Remove recursively	Importance	Ranked selection

3. Embedded Methods

- Feature selection happens during model training.

Common Techniques:

- LASSO (L1 Regularization, useful for feature selection)
- Ridge Regression (L2, balances the weights of features without removing them)
- Decision Trees / Random Forest feature importance

Advantages:

- Balanced approach (accuracy + efficiency)
- Automatically selects important features

Disadvantages:

- Depends on chosen model

Advanced Techniques for feature selection:

- Principal Component Analysis (PCA) (Dimensionality Reduction)
- Variance Threshold Method
- Mutual Information
- Genetic Algorithms (Feature Selection)

PCA (Principal Component Analysis) in Data Analytics

Principal Component Analysis (PCA) is a dimensionality reduction technique used in data analytics to:

- Reduce the number of features (variables)
- Preserve maximum information (variance)
- Transform data into a new coordinate system

It converts original correlated features into new uncorrelated variables called Principal Components (PCs).

Why PCA is Used

- Reduce complexity of data
- Remove redundancy (correlated features)
- Improve visualization (2D/3D plots)
- Speed up machine learning algorithms
- Reduce overfitting

PCA Working in brief:

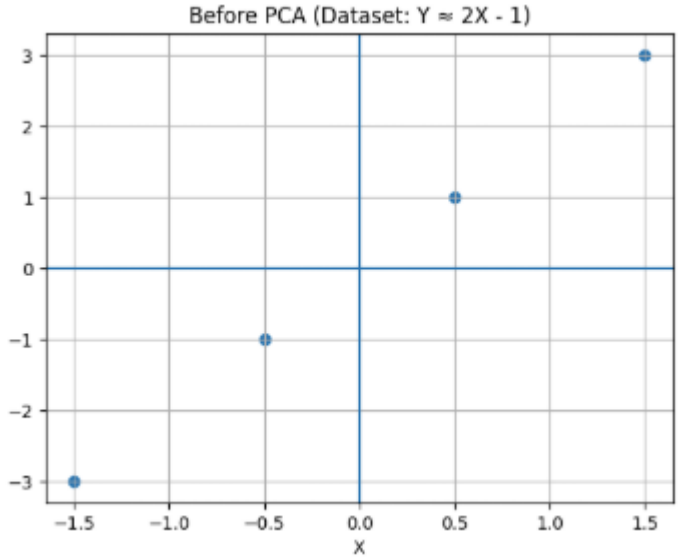
- Standardize the Data
  - Convert features to same scale (mean = 0, variance = 1)
- Compute Covariance Matrix
  - Shows how variables vary together
- Calculate Eigenvalues & Eigenvectors
  - Eigenvectors → direction of principal components
  - Eigenvalues → importance
- Sort Eigenvalues
  - Highest eigenvalue = most important component
- Select Top k Components
  - Choose components that explain most variance
- Transform Data
  - Project original data onto selected components

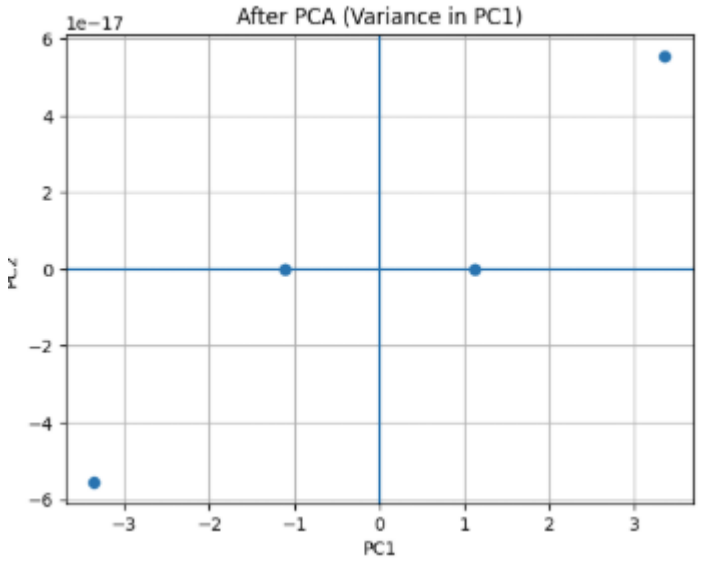
- Example 1:

X	Y
2	3
3	5
4	7
5	9

Steps to follow:

1. Computer Mean
2. Centre the Data
3. Compute covariance matrix
4. Find out Eigen value and Eigen Vector
5. Observe Principal Component equation
6. Transform data, observe projection
7. Interpretation from final transformed data

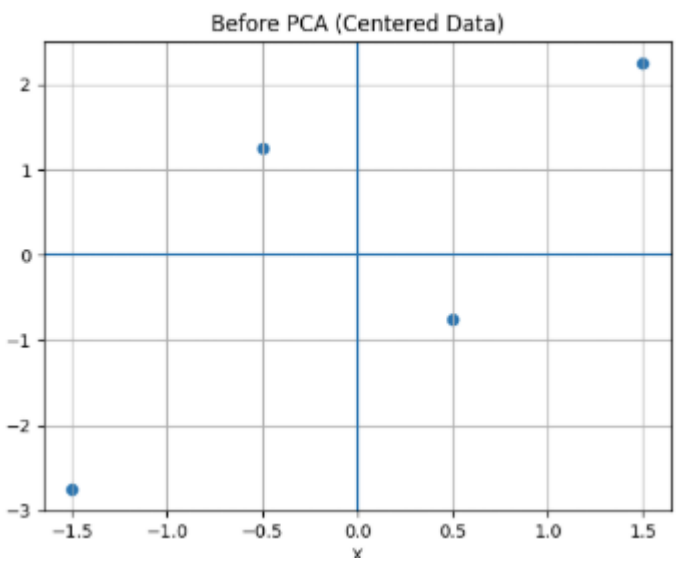

Before PCA (Dataset: Y ≈ 2X - 1)
3
2
1
0
−1
−2
−3
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
X

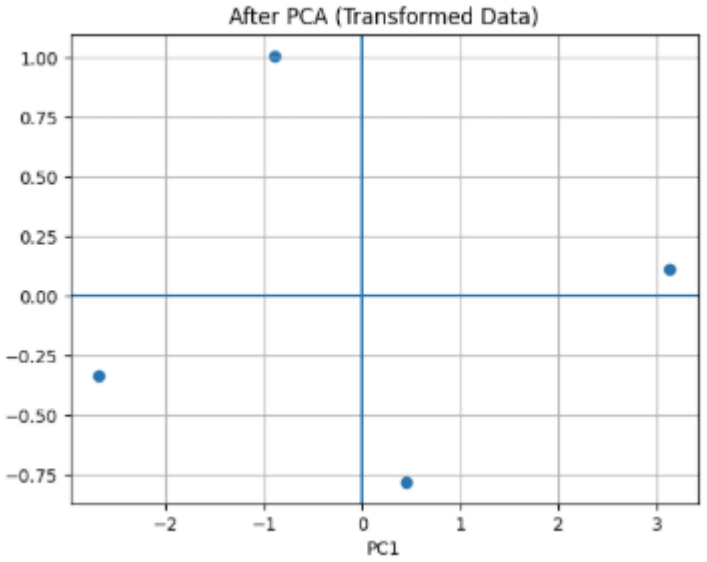

After PCA (Variance in PC1)
1e−17
6
4
2
0
−2
−4
−6
−3
−2
−1
0
1
2
3
PC1

- Example 2:

X	Y
2	1
3	5
4	3
5	6

1. Computer Mean
2. Centre the Data
3. Compute covariance matrix
4. Find out Eigen value and Eigen Vector
5. Observe Principal Component equation
6. Transform data, observe projection
7. Interpretation from final transformed data


Before PCA (Centered Data)
2
1
0
−1
−2
−3
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5


After PCA (Transformed Data)
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−2
−1
0
1
2
3
PC1

Feature Selection vs Feature Engineering

Feature Selection	**Feature Engineering**
Selects existing features	Creates new features
Reduces dimensionality	Expands or transforms data
Removes irrelevant data	Enhances data representation

Conclusion

- Feature engineering is a critical step in data analytics and machine learning.
- Well-engineered features improve model performance significantly.
- It converts raw data into meaningful information that algorithms can use effectively.